Utilización Github Copilot

El framework Spring Boot

- Facilita la construcción de aplicaciones Backend (microservicios), reduciendo la configuración manual de aplicaciones y simplificando la gestión de dependencias

- Adecuado para la creación de aplicaciones N-capas

- Proporciona starters que agrupan una serie de dependencias y aplican configuraciones por defecto

- Se integra con entornos de desarrollo, como eclipse. Spring Tools Suite (ST) es una versión adaptarda de eclipse para desarrollo de aplicaciones con Spring boot

Antonio Martín Sierra

Arquitectura N-capas

- Divide el desarrollo en varias capas, lo que proporciona las siguientes ventajas:
  - Garantiza la separación de responsabilidades
  - Facilita el mantenimiento
  - Facilita el testeo
  - Aplicaciones más escalables
  - Permite cumplir con estándares internos

Antonio Martín Sierra

Capas habituales en un Backend

Capa	Responsabilidad
API / Web	Controladores REST, validaciones de entrada, códigos de estado
Service	Lógica de negocio, orquestación, reglas corporativas
Repository	Acceso a datos, JPA, consultas
Entity / Domain	Modelos persistentes
DTOs	Modelos de entrada/salida
Mapper	Conversión entre DTOs y entidades
Config	Configuración técnica (Swagger, H2, CORS, etc.)

Estructura de paquetes corporativa

- Dentro del curso utilizaremos es estructura de capas / paquetes:
  - com.capgemini.<microservicio>.api
  - com.capgemini.<microservicio>.api.domain
  - com.capgemini.<microservicio>.config
  - com.capgemini.<microservicio>.web
  - com.caixaba.absis.<microservicio>.service
  - com.caixaba.absis.<microservicio>.repository
  - com.caixaba.absis.<microservicio>.entity
  - com.caixaba.absis.<microservicio>.mapper

Antonio Martín Sierra

Enfoque Api First

- Según este enfoque, se diseña primero el contrato de lo que se quiere ofrecer al Backend
- Utilizamos OpenAPI Generator
- Se genera de forma automática:
  - Endpoints
  - Parámetros
  - Modelos
  - Validaciones
  - Código de estado

Antonio Martín Sierra

El papel de Github Copilot

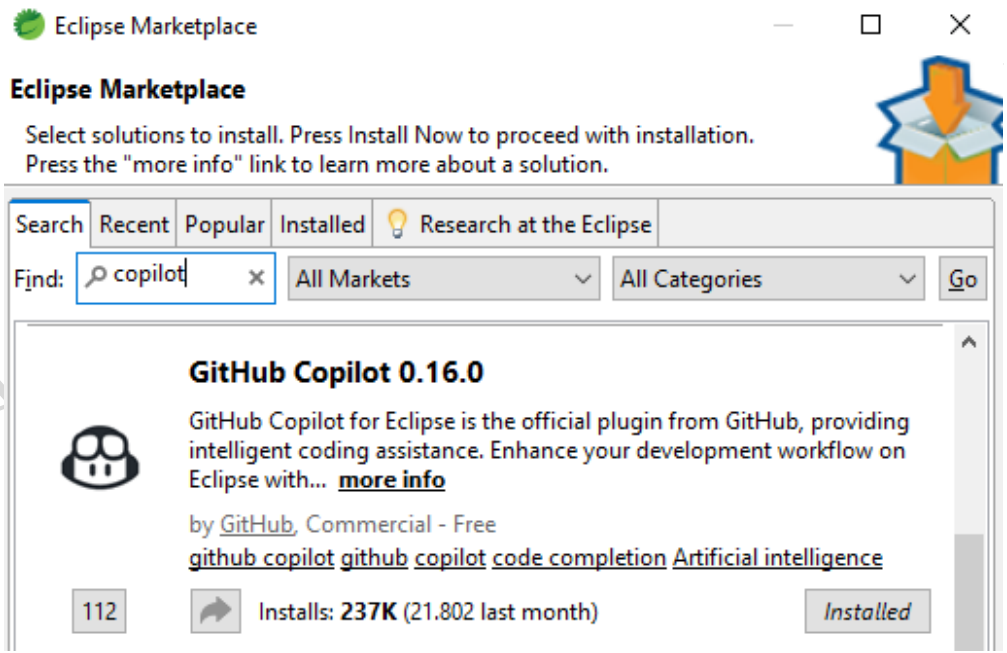
- Copilot no aporta en la generación del API, que es creado automáticamente por OpenAPI generator
- Copilot ayuda en la creación de los elementos manuales:
  - Lógica de negocio
  - Entidades y repositorio
  - Controladores
  - Mappers
  - Tests

Antonio Martín Sierra

El plugin copilot en STS

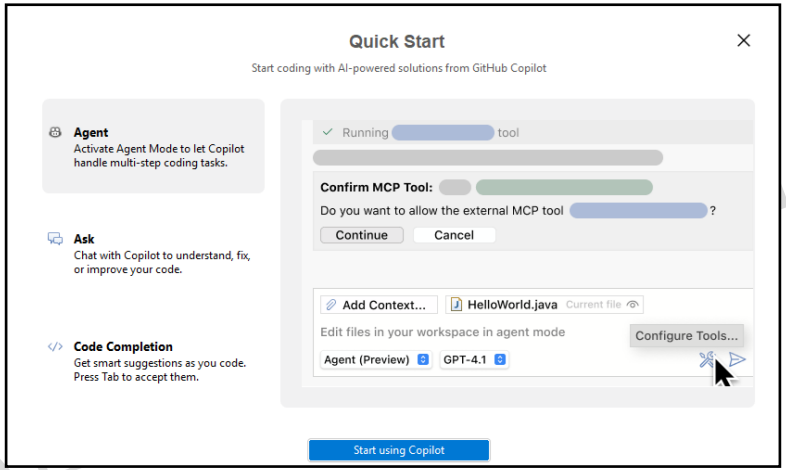
INSTALACIÓN

- **Desde el menú Help->Eclipse Marketplace de STS se busca copilot**

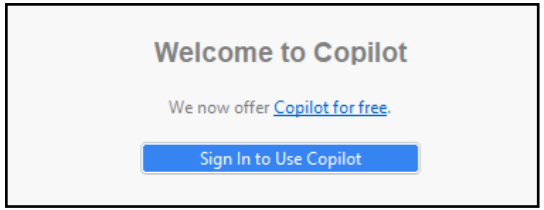


Validar suscripción

- Una vez instalado se debe validar la suscripción pulsando "Start using copilot"

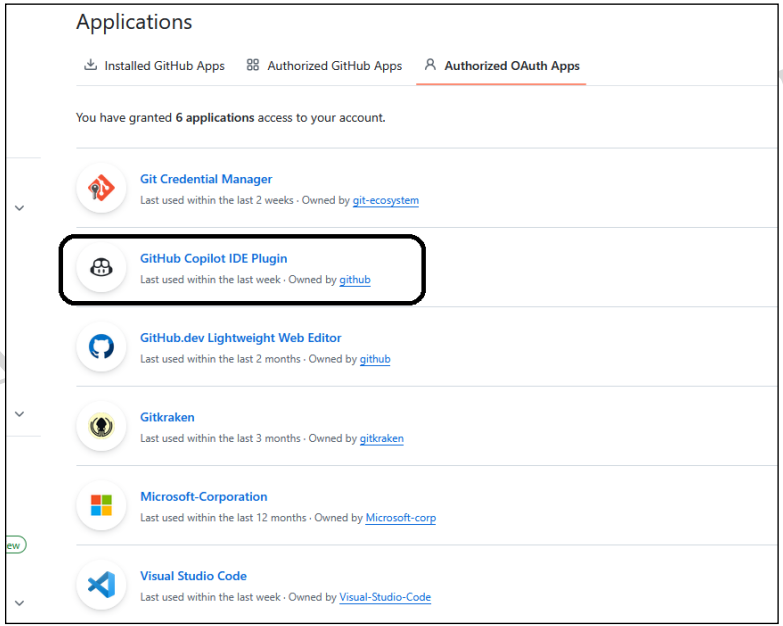


- Después solicitará autenticarnos con copilot:



VERIFICACIÓN

- **Dentro de nuestra cuenta de Github, en settings-> Applications verificamos que el plugin Copitot aparece en la lista de aplicaciones autorizadas con OAuth**



Verificación de código público

- La opción Activar "Block suggestions matching public code", ya no aparece en las últimas versiones del plugin
- El filtrado de coincidencias con código público es automático
- GitHub lo integró directamente en el motor de Copilot para evitar:
  - Sugerencias copiadas de repos públicos
  - Coincidencias largas
  - Fragmentos con riesgo de licencia
  - Así que no te falta nada: el plugin ya lo hace automáticamente

Antonio Martín Sierra

Modos de uso

- **Existen dos modos de uso del plugin GitHub Copilot:**
  - **Modo completions**
  - **Modo chat**
- **Además de estas tenemos el Microsoft Copilot, que es un asistente general, tipo Chat GPT pero mantenido por Microsoft, al que se accede vía Web:**

https://copilot.microsoft.com/

Antonio Martín Sierra

Modo completions

- Se va mostrando sugerencias mientras se escribe. Es similar a un intellisense, pero generativo.

- Funciona en contexto local inmediato, es decir, toma como referencia el archivo actual y cercanos.

- En este modo no hay conversación. Resulta ideal para código repetitivo, patrones conocidos como DTOs, mappers, acelerar tareas mecánicas,...

- El modo *completions* acelara la escritura de código, pero no el diseño. Por tanto, entre sus limitaciones están el hecho de que no entiende el objetivo global de nuestro desarrollo y puede dar lugar a errores de sugerencia

Antonio Martín Sierra

El contexto en completions

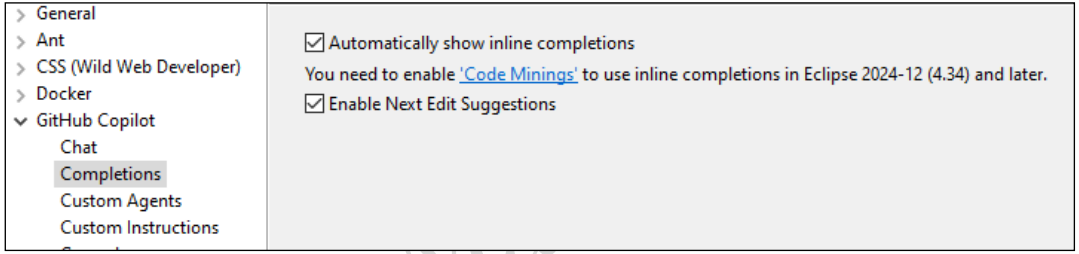
- El contexto del modo completions es el archivo actual, el lenguaje y sintaxis detectada, así como los elementos importados y visibles (clases, métodos, tipos accesibles,..). Depende del IDE, puede usar como contexto otros archivos abiertos

- Para generar la respuesta, en modo completions se envían al exterior fragmentos muy limitados del código cercano e Información técnica (lenguaje, IDE)

Antonio Martín Sierra

Activación/desactivación

- Desde Window->Preferences->Github Copilot->Completions, se puede activar y desactivar la opción de completions en línea



- La opción "Enable Next Edit Suggestions" permite que copilot pueda ofrecer sugerencias de refactorización de código

Antonio Ma

Riesgos legales

- **Se debe evitar código público o bajo licencia.**

- **Ejemplo:**

```
// Implementar un árbol rojo-negro completo
```

- **Proporciona una implementación famosa GPL. La organización puede verse obligada a liberar el código**

- **Ejemplo:**

```
// Implementar el algoritmo de compresión LZ77 igual que zlib
```

- **Copilot puede intentar reproducir partes del código de zlib, que tiene licencia específica. Mejor indicar intención:**

```
//Implementa un algoritmo de compresión basado en ventanas deslizantes
```

Antonio Martín Sierra

Riesgos legales

- **Si copilot añade comentarios sospechosos, desconfiar de la solución.**

- **Ejemplo:**

```
// Based on the implementation from XYZ project
```

- **O:**

```
// Copyright (c) 2018 John Doe
```

- **Cuidado con códigos famosos de stack overflow:**

```
// Deep clone de un objeto en JavaScript
```

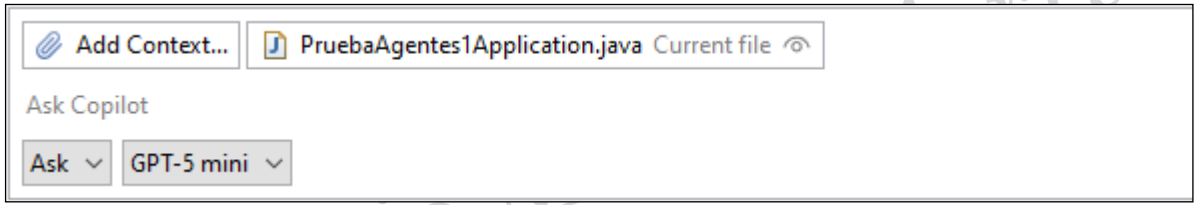
- **Copilot devuelve un famoso código de stackoverflow:**
- **Evitar códigos inseguros y problemáticos**

```
//Login sencillo con usuario y contraseña
```

Antonio Martín Sierra

Modo chat

- En el modo chat, copilot muestra una interfaz conversacional integrada en el IDE,a través de la cual podemos preguntar, pedir explicaciones, refactorizar y generar código



- Mientras completions escribe más rápido el código que ya sabemos hacer, chat nos ayuda con el diseño

El contexto en chat

- El contexto del chat es el archivo actual, archivos abiertos, estructura del proyecto, lenguaje y frameworks detectados, y por supuesto, el historial del chat. Además, puede usar como contexto otro código que le propongamos por el chat:

- En el caso de chat, la información que se envía aGithub para generar la respuesta es el prompt completo, código relevante (archivo, selección, a veces varios archivos) y parte del historial del chat.

Antonio Martín Sierra

Modos de funcionamiento del chat

- **En el modo chat podemos trabajar con de tres formas:**

Ask

Agent

Plan

Antonio Martín Sierra

Modo Ask

- Para preguntas rápidas y consultas. Es el más directo y conversacional. Ideal para conocer el código o aprender sobre algún uso específico

- Ni modifica archivos, ni ejecuta acciones ni crea nuevas clases

- Se utiliza para pedir explicaciones, sugerencias, entender y decidir

- Es el modo más utilizado del chat

Antonio Martín Sierra

Comandos en modo Ask

- Se pueden ejecutar algunos comandos predefinidos para obtener información sobre la clase que aparece en el editor (en STS no detecta código seleccionado). Entre estos:
  - /explain. Proporciona una explicación sobre lo que aparece en el editor
  - /fix. Detecta posibles errores o mejoras y ofrece soluciones. No mejora código.
  - /doc. Genera documentación Javadoc para el archivo actual. No modifica el archivo, nos muestra como sería el código resultante

Antonio Martín Sierra

Comandos en modo Ask

- **/test. Genera test unitarios para el componente abierto en el editor. No crea el archivo físicamente, nos muestra el código de la clase de test**

- **/optimize. Te muestra una optimización del código actual más limpia. Se le puede pedir que nos explique las modificaciones:**

  /optimize y explicame los cambios. No hagas tests

- **/review. Realiza una revisión de calidad del código. Detecta riesgos, propone mejoras de estilo,..**

- **/example. Muestra ejemplos de uso, aunque en STS no tiene un handler específico y hace lo mismo que /review**

Antonio Martín Sierra

Modo Agent

- Es el más potente, actúa como un asistente de desarrollo. Es útil para encargarle a copilot tareas reales y complejas, como crear la estructura de un proyecto completo, generar test, ejecutar comandos, modificar configuraciones,....
- Puede crear y modificar archivos, siempre con autorización previa
- No ejecuta código, ni crea proyectos nuevos ni instala dependencias automáticamente
- Hay que usarlo con precaución por los cambios que realiza

Antonio Martín Sierra

Modo Plan

- Se utiliza para realizar tareas que requieren varios pasos: "Añademe al proyecto seguridad basada en JWT..."
- Define un plan de trabajo con una serie de fases
- Durante el proceso puede realizar preguntas
- No hace modificaciones por si solo. Al final, aparece un botón por si queremos que se ejecute el plan. Es entonces cuando se realizan los cambios

Antonio Martín Sierra

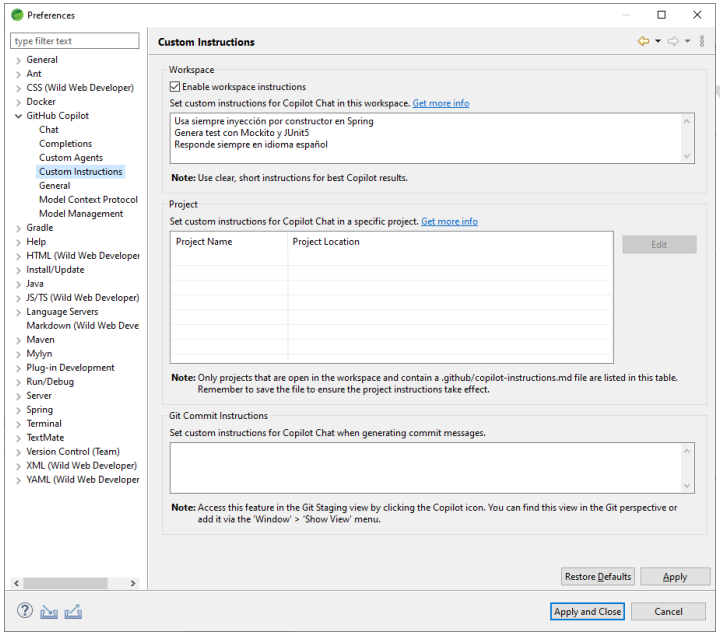
Diferencias Ask/Agent

Característica	ASK	AGENT
Explica código	✔	✔
Genera ejemplos	✔	✔
Sugiere mejoras	✔	✔
Genera tests	✔	✔
Modifica archivos	✘	✔
Crea archivos nuevos	✘	✔
Refactoriza varios archivos	✘	✔
Usa contexto ampliado	✔	✔
Necesita confirmación	✘	✔
Puede romper cosas	✘	✔ (si no se revisa)

Custom instructions

- **Se utilizan para inyectar un texto fijo que Copilot utilizará en todos sus prompts en modo Ask. Se accede:**

Preferences → GitHub Copilot → Custom Instructions



n Sierra

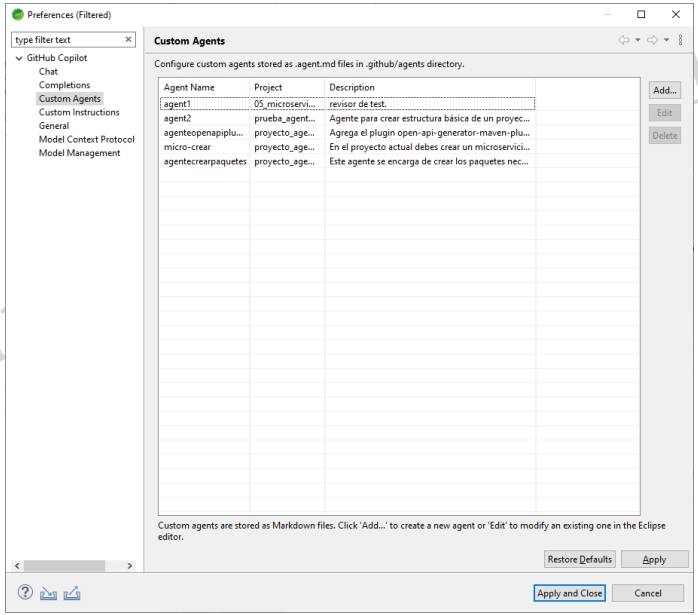
Custom instructions

- Se definen tres secciones:
  - Workspace: Aquí indicamos instrucciones generales para todos los proyectos del Workspace
  - Project. Son instrucciones que se deben seguir para un proyecto específico
  - Get Commit Instructions. Se trata de instrucciones que copilot debe seguir en caso de que lo utilicemos para generar mensajes de commits automáticos cuando se hacen subidas a git desde STS

Antonio Martín Sierra

Agentes personalizados

➢**Desde el chat de copilot, seleccionando la opción "Configure Agents", se accede a la opción "Custom Agents" que permite crear agentes personalizados:**



Crear agentes personalizados

- **Tras crearlo y asignarle un nombre, en el editor se indica una descripción, herramientas a utilizar e instrucciones a seguir:**

```
---
description: En el proyecto actual debes crear un microservicio a partir de la interfaz y modelos de datos
creados en {{microname}}.api, siendo "microcrear" el nombre del parámetro proporcionado por el usuario
tools: ['run_in_terminal', 'manage_todo_list', 'get_errors', 'run_subagent', 'replace_string_in_file', 'create_file',
'get_terminal_output', 'list_dir', 'file_search', 'grep_search', 'validate_cves', 'insert_edit_into_file', 'read_file']
---
Tu tarea es crear el microservicio siguiendo exactamente estas reglas:

PARÁMETRO OBLIGATORIO:
- El usuario debe indicar el nombre del paquete donde se encuentran los componentes generados por Open API. El nombre
de ese parámetro será <microname>
- Estos componentes están en target/generated-sources/openapi/src/main/java/{{microname}}.api

1.- Para la capa de datos utilizaremos Spring Data JPA y base de datos H2. Las entidades se crean en
com.caixaba.{{microname}}.entity
y el repositorio en com.caixaba.{{microname}}.repository
2.- Revisa el pom.xml por si fuera necesario alguna dependencia adicional
3.- Implementa la capa de lógica de negocio con interfaz y clase de implementación en el paquete
com.caixaba.{{microname}}.service
:
```

- **Se pueden especificar parámetros**
- **Para ejecutar se selecciona en la lista del chat**